# LAB EXERCISE: 3

# CineGraph: Movie Question Answering using Graph RAG

| | | | |
|---|---|---|---|
| Student Name | Aditi S Yaranal | USN | 1GA23AI003 |
| Semester / Department | 7th Semester / AIML | Date | 7 October 2026 |
| Institution | Global Academy of Technology | Course | AML23702 |

## Aim / Objective

To design and implement a small retrieval-augmented generation application that uses the Neo4j Movies dataset as its knowledge graph, retrieves movie, actor and director information through Cypher queries, and uses a local LLM to generate natural-language answers.

## Expected Outcomes Addressed

- Load and use the Neo4j movie graph: the local database contains 38 movies, 133 people and 253 relationships.
- Convert user questions into graph queries: a rule-based intent parser selects one of six parameterized Cypher templates.
- Retrieve relevant movie and actor data: cast, directors, filmographies and release details are returned as evidence rows.
- Generate grounded answers: Qwen receives graph evidence; the interface exposes sources and flags missing or invalid citation IDs. Semantic grounding remains imperfect.
- Support movie QA and recommendations: the web application answers supported questions and ranks related movies by shared actors.

## Tools & Technologies Used

| Category | Tool / Library | Version | Purpose |
|---|---|---|---|
| Language | Python | 3.12.14 | Backend and evaluation |
| Graph database | Neo4j Community | 2026.09.0 | Movies graph and Cypher |
| Database driver | neo4j Python driver | 6.4.0 | Parameterized retrieval |
| Web framework | Flask | 3.1.3 | Local web API |
| LLM / inference | Qwen2.5 3B Instruct / llama.cpp | Q4_K_M / b11443 | Local answer generation |
| Frontend | HTML, CSS, JavaScript, SVG | Web standards | Responsive UI and graph |

## Dataset / Corpus Description

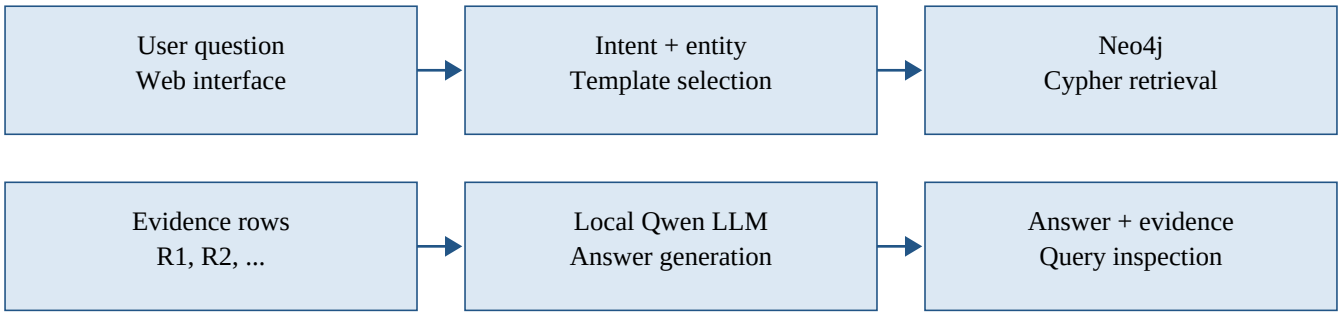
Source: the official neo4j-graph-examples/movies repository. The import script is included as data/movies.cypher. Its source and SHA-256 are recorded in data/provenance.json. It is loaded into a dedicated local Neo4j instance; the initialization script imports only when no Movie nodes are present.

| Data element | Description / measured count |
|---|---|
| Movie nodes | 38; properties: title, released, tagline |
| Person nodes | 133; properties: name, born |
| Relationships | 253 total, including ACTED_IN, DIRECTED, WROTE, PRODUCED, REVIEWED and FOLLOWS |
| Relationships queried | ACTED_IN (with roles) and DIRECTED |
| Matching / constraints | Case-insensitive exact names; official title and person-name uniqueness constraints |

Sample record: Movie {title: "The Matrix", released: 1999, tagline: "Welcome to the Real World"}. Sample relation: (Keanu Reeves)-[ACTED_IN {roles: ["Neo"]}]->(The Matrix).

The corpus is an educational sample, not a complete film catalogue. For example, it includes Emil Eifrem in The Matrix cast. Grounding means fidelity to the supplied graph, not independent verification of real-world film facts.

## System Architecture / Pipeline Design




- Module 1 - Input handling: validate an English question of up to 500 characters and identify the movie/person.
- Module 2 - Retrieval: bind the entity to a fixed read-only query and retrieve bounded rows through the Neo4j driver.
- Module 3 - Evidence preparation: assign row IDs; detect partial results by fetching one extra row.
- Module 4 - Generation: send facts to the local model. Empty results skip generation; model failures preserve evidence.
- Module 5 - Presentation: show answers, source rows, Cypher, movie search and the relationship graph.

Core schema: (Person)-[ACTED_IN {roles}]->(Movie); (Person)-[DIRECTED]->(Movie). The LLM does not write or execute Cypher.

# Backend Implementation

## 6.1 Approach / Algorithm

The parser recognizes cast, director, actor_movies, director_movies, movie and recommend intents. It uses case-insensitive patterns and exact entity matching. Unsupported phrasing returns examples instead of guessing. Query values are parameterized, so user text cannot change the query structure.

For recommendations, traverse from the seed movie to shared actors and then to other movies. Exclude the seed, count distinct shared actors and sort by descending count, with title breaking ties. This score measures cast overlap, not popularity or user preference.

## 6.2 Key Code Snippets

1. Question parsing and template selection

```
for intent, pattern in PATTERNS:
    match = re.fullmatch(pattern, q, flags=re.IGNORECASE)
    if match:
        entity = match.group(1).strip().strip('"').strip()
        if entity:
            return intent, entity
```

2. Read-only parameterized graph retrieval

```
records, _, _ = driver.execute_query(
    QUERIES[intent],
    parameters_={"entity": entity, "limit": limit + 1},
    database_=database, routing_="r")
rows = [record.data() for record in records]
return rows[:limit], len(rows) > limit
```

3. Local model request (excerpt; configuration supplies the endpoint)

```
payload = {
    "messages": [
        {"role": "system", "content": local_system},
        {"role": "user",
         "content": "Question: " + question + "\nFacts:\n" + facts}
    ],
    "max_tokens": 650, "temperature": 0, "stream": False
}
# POST payload to llama.cpp /v1/chat/completions
```

The application checks whether generated citation IDs exist in the evidence range. This check does not prove that each cited row supports the claim. A visible warning is shown when citations are missing or misnumbered.

# Frontend / UI Implementation

## 7.1 Framework Choice

Flask serves the local API and static HTML/CSS/JavaScript frontend. SVG renders the movie relationship graph. The interface is responsive and was checked at desktop and 390-pixel mobile widths.

## 7.2 Interface Features

- Ask CineGraph: typed questions, sample prompts, generated answers, elapsed time, evidence tables and Cypher.
- Movie collection: search 38 titles and select a movie to ask about it.
- Knowledge graph: choose a movie to display its cast and directors.
- About the project: architecture, limitations and a link to the report.

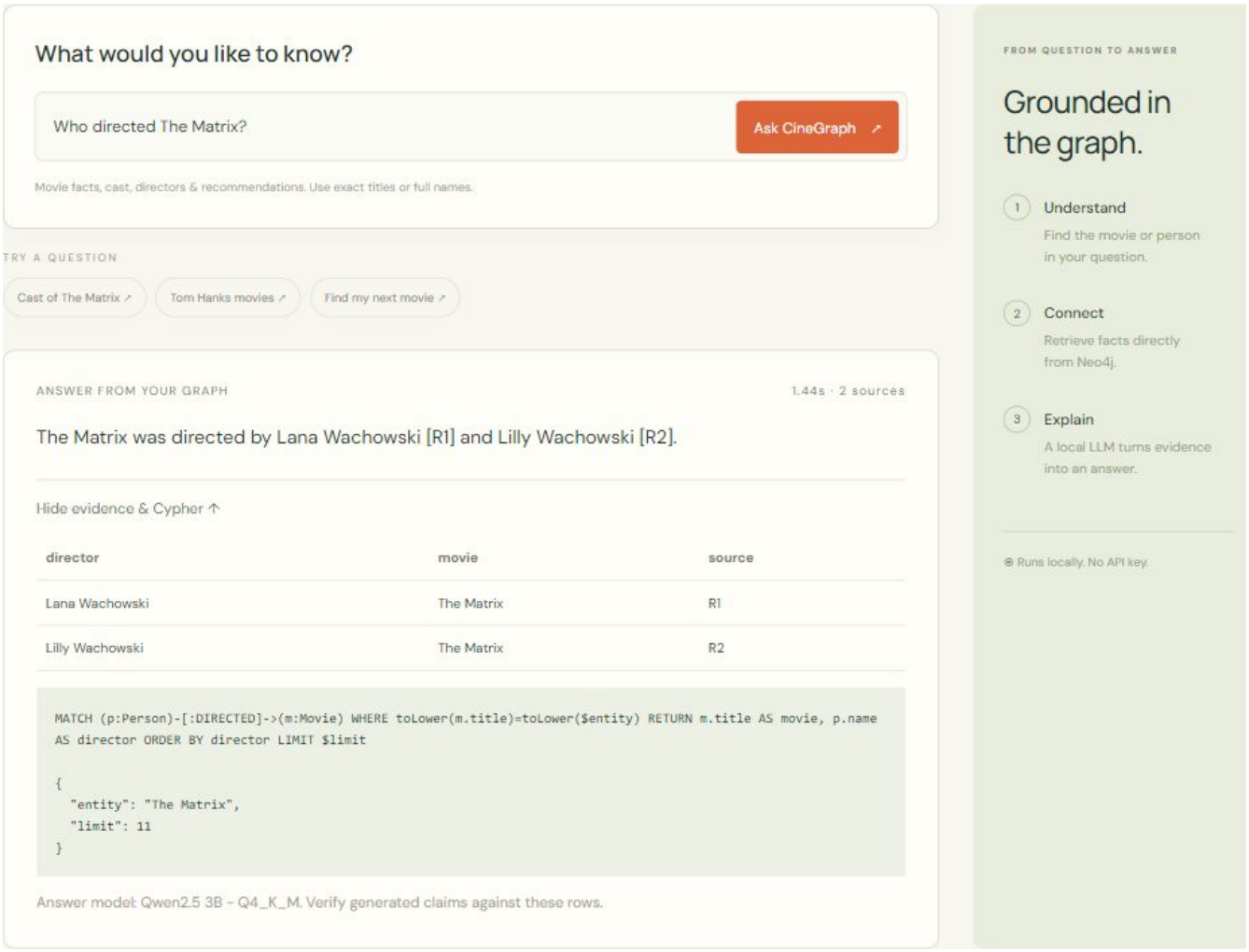



Figure 1. Working frontend: a live director answer with its graph evidence (captured 7 October 2026).

Run locally at http://127.0.0.1:8765. Double-click Launch CineGraph.cmd to restart the database, model and web server. No paid LLM API key is required.

## Sample Input & Output

| Input | Component | Observed output |
|---|---|---|
| Who directed The Matrix? | director | The Matrix was directed by Lana Wachowski [R1] and Lilly Wachowski [R2]. |
| When was The Matrix released? | movie | The Matrix was released in 1999. [R1] |
| Movies with Tom Hanks | actor_movies | Ten movie rows including Apollo 13 (1995); partial-list notice displayed. |
| Recommend movies like The Matrix | recommend | Eight retrieved candidate movies; shared actor counts and names available as evidence. |
| Tell me about __missing_movie__ | No evidence | No matching evidence found; LLM not called. |
| Who won an Oscar? | Unsupported input | HTTP 400 with supported question examples. |

## Results (Deliverable-Specific Outputs)

### Neo4j Database Instance

Local Neo4j Community is configured at bolt://127.0.0.1:7689; its Browser interface is at http://127.0.0.1:7479. The loaded graph has 171 nodes and 253 relationships. Credentials are private and excluded from the repository.

### Cypher Query Templates

Six complete templates are included in queries.json and cypher_templates.cypher. All application templates are read-only and use bound parameters.

### Retrieved Recommendation Evidence

| Movie | Year | Shared actors / score |
|---|---|---|
| The Matrix Reloaded | 2003 | Hugo Weaving, Laurence Fishburne, Carrie-Anne Moss, Keanu Reeves / 4 |
| The Matrix Revolutions | 2003 | Hugo Weaving, Laurence Fishburne, Carrie-Anne Moss, Keanu Reeves / 4 |
| Cloud Atlas | 2012 | Hugo Weaving / 1 |

Other deliverables: a working web/CLI prototype, setup and launch scripts, this report, offline_test_results.txt and output/live_results.json with actual questions, answers, source rows, parameters and timings.

## Evaluation / Analysis

### Performance Evaluation

| Check | Observed result | Interpretation |
|---|---|---|
| Unit / web tests | 12 / 12 passed | Parser, parameters, errors, payloads, routing and PDF endpoint |
| Live Neo4j checks | 7 / 7 passed | Six retrieval intents plus absent entity |
| Live website acceptance | 8 / 8 passed | Expected retrieval and request behavior; successful model calls |
| Evidence-backed latency | 1.59-14.67 s; mean 6.89 s | Six requests in one local run, not a benchmark |
| Semantic groundedness | Not scored quantitatively | Known citation omissions and unsupported commentary remain |

### Interpretation

Passing retrieval tests does not establish perfect answer quality. The saved cast answer omitted citations, and the Nora Ephron answer added genre/style commentary absent from the retrieved rows. The interface warns about missing citations; human checking is still needed for semantic correctness. No precision, recall or F1 score is invented.

## Challenges Faced & Solutions

- Java runtime conflict: Neo4j initially selected a stripped Android runtime without the required vector module. The launcher now resolves the full installed JDK for the project.
- Weak model adherence: the initial 1.5B model produced poor citations and vague recommendations. A local 3B model and simpler evidence prompts improved readability; the report retains observed limitations.
- Incomplete or unsupported questions: exact matching can return no data. Explicit error messages and example question forms guide the user; empty evidence prevents an LLM call.
- Private runtime management: credentials and large model/database files are stored under .runtime and excluded from Git. Reproduction scripts download the required runtimes.

## Conclusion

CineGraph implements a small graph RAG application with a real Neo4j Movies backend, controlled Cypher retrieval, local LLM generation and an inspectable web interface. The exercise demonstrates graph modelling, intent-based retrieval and evidence-grounded prompting. Future improvements include alias resolution, semantic citation verification and a larger held-out question set.

## Project Repository (GitHub Link)

| Field | Submission details |
|---|---|
| GitHub Repository URL | https://github.com/Aditisy/Cinigraph |
| Repository Visibility | Public |
| Branch Used for Submission | main |
| Student / USN | Aditi S Yaranal / 1GA23AI003 |
| Date of Submission | 7 October 2026 |

The repository contains backend/frontend source, the official data import script, six Cypher templates, configuration examples, requirements, setup/run instructions, test evidence and the final report. Private credentials, local database files and model/runtime binaries are excluded. They can be recreated with the included setup scripts.

### Reproduction Commands

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe setup_local.py
Copy-Item config.local.example.json config.json
powershell -ExecutionPolicy Bypass -File .\start_local.ps1
```

Prerequisites: Windows, Python 3.10 or later, a supported full Java 21/25 JDK and sufficient disk space for runtime/model downloads. The Flask server is for local review; the GitHub repository itself does not host the running Neo4j/LLM application.

## References


- Lab Exercise with Rubrics (2).pdf: Exercise 3 on page 2 and assessment rubric on pages 4-5.
- ANLP_Lab1_report013.pdf: user-provided report-format reference; section order and evaluation-sheet styling adapted for Exercise 3.
- Neo4j Movies dataset: https://github.com/neo4j-graph-examples/movies
- Neo4j Python driver: https://neo4j.com/docs/python-manual/current/query-simple/
- Neo4j Windows installation: https://neo4j.com/docs/operations-manual/current/installation/windows/
- llama.cpp runtime: https://github.com/ggml-org/llama.cpp
- Qwen model: https://huggingface.co/Qwen/Qwen2.5-3B-Instruct-GGUF
- Flask documentation: https://flask.palletsprojects.com/

# ADVANCE NLP (AML23702)

# LAB EXERCISE EVALUATION SHEET

| | |
|---|---|
| Student Name | Aditi S Yaranal |
| USN | 1GA23AI003 |
| Lab Exercise No. | 3 |
| Experiment Title | CineGraph: Movie Question Answering using Graph RAG |
| Semester / Department | 7th Semester / AIML |
| Institution | Global Academy of Technology |

## Marks Awarded

| Sl. No. | Evaluation Criteria | Max Marks | Marks Obtained |
|---|---|---|---|
| 1 | Problem Understanding & Scope | 5 | |
| 2 | Pipeline Design & Methodology | 10 | |
| 3 | Implementation Correctness | 10 | |
| 4 | Use of NLP Tools / Models | 5 | |
| 5 | Output Quality / Task Performance | 10 | |
| 6 | Analysis & Interpretation | 5 | |
| 7 | Documentation & Deliverables | 2.5 | |
| 8 | Presentation & Reproducibility | 2.5 | |
| | Total | 50 | |

### Overall Comments

| | |
|---|---|
| Date of Submission | 7 October 2026 |
| Signature of Faculty | |